

Trabajo Práctico Especial 2026/1

Abstract

Este documento describe el Trabajo Especial de la materia Protocolos de Comunicación para la cursada del primer cuatrimestre del año 2026.

En su ejecución los alumnos DEBEN demostrar habilidad para la programación de aplicaciones cliente/servidor con sockets, la comprensión de estándares de la industria, y la capacidad de diseñar protocolos de aplicación.

Terminología

Las palabras clave "DEBE", "NO DEBE", "OBLIGATORIO", "DEBERÁ", "NO DEBERÁ", "DEBERÍA", "NO DEBERÍA", "RECOMENDADO", "PUEDE" y "OPCIONAL" en este documento serán interpretadas como se describe en el [RFC 2119](#).

1. Requerimientos Funcionales

El objetivo del trabajo es implementar un servidor proxy para el protocolo SOCKS v5 ([RFC1928](#)).

El servidor DEBE:

1. Atender a múltiples clientes en forma concurrente y simultánea (al menos 500).
2. Soportar autenticación usuario / contraseña ([RFC1929](#)).
 - Nota: por más que el RFC de SOCKS v5 requiera soportar el método de autenticación GSSAPI, esto no es un requisito para este TP
3. Soportar de mínima conexiones salientes a servicios TCP a direcciones IPv4, IPV6, o utilizando FQDN que resuelvan cualquiera de estos tipos de direcciones.
4. Ser robusto en cuanto a las opciones de conexión (si se utiliza un FQDN que resuelve a múltiples direcciones IP y una no está disponible debe intentar con otros).
5. Reportar los fallos a los clientes usando toda la potencia del protocolo.
6. Implementar mecanismos que permitan recolectar métricas que ayuden a monitorear la operación del sistema:
 - a. Cantidad de conexiones históricas
 - b. Cantidad de conexiones concurrentes
 - c. Cantidad de bytes transferidos
 - d. Cualquier otra métrica que considere oportuno para el entendimiento del funcionamiento dinámico del sistema

Las métricas PUEDEN ser volátiles (si se reinicia el servidor las estadísticas pueden perderse).

7. Diseñar e implementar un protocolo de monitoreo que permita manejar usuarios y cambiar la configuración del servidor en tiempo de ejecución sin reiniciar el servidor.
 - Las diferentes implementaciones PUEDEN decidir disponibilizar otros cambios en tiempo de ejecución de otras configuraciones (memoria utilizada en I/O, timeouts, etc).
 - Esto no es una extensión del protocolo SOCKS, es un nuevo protocolo escuchando en otro socket pasivo en otro puerto dentro del mismo programa.
8. Implementar un registro de acceso que permita a un administrador entender los accesos de cada uno de los usuarios. Pensar en el caso de que llega una queja externa y el administrador debe saber quién fue el que se conectó a cierto sitio web y cuando.
9. Realizar *graceful shutdown*: manejar señales SIGTERM y SIGINT para realizar un apagado controlado. Al recibir la señal, el servidor DEBE dejar de escuchar por nuevas conexiones y esperar a que todas las conexiones existentes terminen antes de apagarse. Una segunda señal PUEDE apagarlo forzosamente.
10. [SOLO PARA SEGUNDA ENTREGA] monitorear el tráfico y generar un registro de credenciales de acceso (usuarios y passwords) de forma similar a ettercap por lo menos para protocolo POP3.

2. Requerimientos No Funcionales

Adicionalmente, la implementación DEBE:

1. Estar escritos en el lenguaje de programación C, específicamente con la variante C11 (ISO/IEC 9899:2011).
2. Utilizar sockets en modo no bloqueante multiplexada.
3. Tener en cuenta todos los aspectos que hagan a la buena performance, escalabilidad y disponibilidad del servidor. Se espera que se maneje de forma eficiente los flujos de información (por ejemplo no cargar en memoria mensajes muy grandes, ser eficaz y eficiente en el intérprete de mensajes). El informe DEBE contener información sobre las pruebas de estrés. Algunas preguntas interesantes a responder son:
 - ¿Cuál es la máxima cantidad de conexiones simultáneas que soporta?
 - ¿Cómo se degrada el throughput?
4. Seguir los lineamientos de IEEE Std 1003.1-2008, 2016 Edition / Base definitions / 12. Utility Conventions (<https://pubs.opengroup.org/onlinepubs/9699919799/nframe.html>) a menos que se

especifique lo contrario: Esto se refiere a cómo manejar argumentos de línea de comandos, parámetros, etc.

5. Deberá documentar detalladamente el protocolo de monitoreo y configuración e implementar una aplicación cliente.
 - El cliente para el protocolo de monitoreo puede usar I/O bloqueante dada su simpleza.
 - El cliente de monitoreo debe brindarle al usuario una forma cómoda de monitorear su servidor desde la terminal. No se acepta diseñar un protocolo de texto y que el cliente sea netcat. Ejemplo: `client add-user pablito pass1234`.
 - Decidir ustedes cómo realizan el manejo de autenticación.
6. Deberá ser compilable utilizando el comando `make` y proveer un Makefile.
7. Tanto la aplicación servidor, como la aplicación cliente de configuración/monitoreo DEBERÁN manejar los argumentos de línea de comandos de cierta forma uniforme (por ejemplo `-p <puerto>` podría especificar el puerto utilizado para el protocolo de configuración/monitoreo). Una implementación del parsing de argumentos será publicada en el campus.
8. Si bien los programas son pequeños podrá utilizar librerías o archivos (fragmento de código) desarrollados por terceros siempre que se cumplan los siguientes requisitos:
 - a. La librería o fragmento NO DEBE resolver las cuestiones de fondo del Trabajo Práctico.
 - b. La librería o fragmento DEBE tener una licencia aprobada por la Open Source Initiative (<https://opensource.org/licenses>).
 - c. El uso de la librería o fragmento DEBE ser aprobada por la Cátedra.

Para lograr la aprobación un alumno del grupo DEBE publicar una secuencia en el foro de discusión del trabajo práctico. La secuencia DEBE describir todos aquellos datos que permitan identificar a la librería (por ejemplo la versión); su licencia de esta forma justificando porqué es válido su uso; y el propósito de su inclusión. En caso de que sea un fragmento de código debe adjuntarse. Está permitido utilizar código publicado por los docentes durante la cursada actual, siempre que se atribuya correctamente.

9. A veces existirán ambigüedades en las especificaciones o múltiples formas en cómo se puede resolver o implementar un problema particular. Por ser una materia de ingeniería se espera que los alumnos tomen decisiones de diseño razonables en estos casos. Los alumnos pueden basar sus decisiones en lo que conoce de antemano de la tarea y en los objetivos enumerados en este documento o demás enunciados. Los docentes pueden darle consejos sobre las ventajas y desventajas de cada decisión, pero los alumnos son los que en última instancia las toman.

Evaluación

La realización del Trabajo Práctico es una actividad grupal. La calificación es de carácter grupal; pero si hay evidencias de que un alumno de un grupo no participó en la elaboración, o éste no puede defender o demostrar su participación, entonces el alumno no podrá aprobar el Trabajo Práctico. Se espera transparencia en el desarrollo del trabajo (entregar el repositorio git).

Cada grupo DEBE entregar todo el material necesario para poder reproducir el Trabajo Práctico. Como mínimo DEBE contener:

- a. Un informe en formato PDF ([RFC3778](#)) o text/plain (con codificación UTF-8) que contenga al menos las siguientes secciones (respetando el orden):
 1. Índice
 2. Descripción detallada de los protocolos diseñados y aplicaciones desarrolladas.
 3. Problemas encontrados durante el diseño y la implementación.
 4. Limitaciones de la aplicación.
 5. Posibles extensiones.
 6. Conclusiones.
 7. Ejemplos de prueba.
 8. Guía de instalación detallada y precisa. No es necesario desarrollar un programa instalador.
 9. Instrucciones para la configuración.
 10. Ejemplos de configuración y monitoreo.
 11. Documento de diseño del proyecto (que ayuden a entender la arquitectura de la aplicación).
- b. Códigos fuente y archivos de construcción
- c. Un archivo README en la raíz que describa al menos:
 1. la ubicación de todos los materiales previamente enumerados
 2. El procedimiento necesario para generar una versión ejecutable de las aplicaciones
 3. La ubicación de los diferentes artefactos generados
 4. Cómo se debe ejecutar las diferentes artefactos generados (y sus opciones)

La entrega se realizará por Campus ITBA en la asignación creada para ello con una fecha de entrega. Se DEBE entregar un tarball o zip que sea el producto de clonar el

repositorio GIT (por lo tanto, el repositorio GIT DEBE contener todos los materiales de entrega), y su historia.

Una vez realizada la entrega los grupos DEBERÁN mostrar el correcto funcionamiento del sistema con casos de prueba provisto por los equipos y provistos ese día por la Cátedra.

Para aprobar el Trabajo Práctico se DEBE cumplir TODAS las siguientes condiciones:

- El material entregado DEBE estar completo (por ejemplo no se puede corregir si falta el informe o archivos de código fuente)
- Se utilizan únicamente las librerías permitidas para los usos definidos.
- DEBE ser correcta las cuestiones de entradas/salida no bloqueante. Por ejemplo las lecturas, escrituras y el establecimiento de nuevas conexiones DEBEN ser mediante suscripciones y no bloquearse, multiplexando conexiones en un único thread.
 - El único uso permitido de múltiples threads es para la resolución de nombres de dominio con [getaddrinfo](#). Este thread NO DEBE realizar ninguna otra operación de I/O, solo devuelve el resultado al thread principal (despertándolo por ejemplo mediante una señal).
 - Considerar usar [getaddrinfo_a](#) para poder evitar el uso de threads completamente.
- DEBE ser correcta las cuestiones relacionadas a la lectura/escrituras parciales.
- Sumar CUATRO puntos de calificación sobre DIEZ puntos posibles.

Se aceptarán entregas tardías entre 0 horas (inclusive) y 24 horas (exclusivo) luego de la fecha límite de entrega, pero la calificación no podrá exceder de CUATRO puntos.